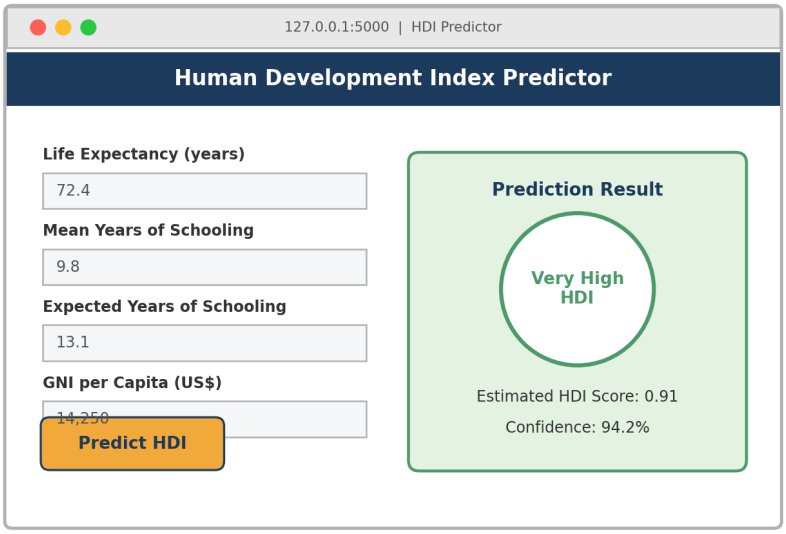
FINAL PROJECT REPORT

Human Development Index (HDI) Prediction using Machine Learning

A Data-Driven Approach to Classifying Human Development Levels of Countries



Domain	Machine Learning / Data Science
Core Libraries	Python, Scikit-learn, NumPy, Pandas, Matplotlib, Seaborn
Deployment	Flask Web Framework
Environment	Anaconda

Table of Contents

1. Introduction

1.1 Project Overview

The Human Development Index (HDI) is a statistical composite index that measures a country's average achievement in three basic dimensions of human development: a long and healthy life, access to knowledge, and a decent standard of living. It is calculated using indicators such as life expectancy at birth, mean years of schooling, expected years of schooling, and Gross National Income (GNI) per capita (PPP). Based on these composite scores, countries are ranked into four tiers — **Very High**, **High**, **Medium**, and **Low** human development.

This project builds a Machine Learning based system that predicts the HDI category of a country given its socio-economic indicators. The solution uses supervised classification algorithms trained on historical HDI data and is deployed as an interactive Flask web application, allowing users to input indicator values and instantly receive a predicted human development classification along with an estimated HDI score.

1.2 Purpose

The HDI was developed to emphasize that people and their capabilities should be the primary measure of a country's development, rather than economic growth alone. It provides a broader perspective on development by considering health, education, and living standards together. The index is widely used by governments, researchers, policymakers, and international organizations to evaluate progress, compare countries, and identify areas requiring improvement.

The purpose of this project is to:

- Automate the process of estimating a country's human development level from raw socio-economic indicators.
- Provide instant, consistent, and explainable predictions instead of slow manual analysis.
- Help policymakers and researchers quickly identify development gaps and prioritise interventions.
- Demonstrate an end-to-end machine learning pipeline — from data collection to a deployed web application.

2. Ideation Phase

2.1 Problem Statement

Governments, researchers and international organisations rely on the Human Development Index to evaluate a nation's progress. However, computing and interpreting the HDI manually from raw indicator data is time-consuming, requires statistical expertise, and does not scale well when comparing many countries or regions simultaneously. There is a need for an intelligent, easy-to-use system that can take basic socio-economic indicators as input and automatically classify the level of human development, enabling faster and more consistent decision-making.

Problem Statement Template	
I am (user)	a policymaker / researcher / student analysing socio-economic data
I'm trying to	quickly assess a country's or region's level of human development
But	manual computation of HDI from raw indicators is slow and error-prone
Because	it requires normalising multiple indices and statistical interpretation
Which makes me feel	frustrated by delayed, inconsistent development insights

2.2 Empathy Map Canvas

The empathy map below captures the perspective of a typical end user (a policymaker or researcher) of the system.

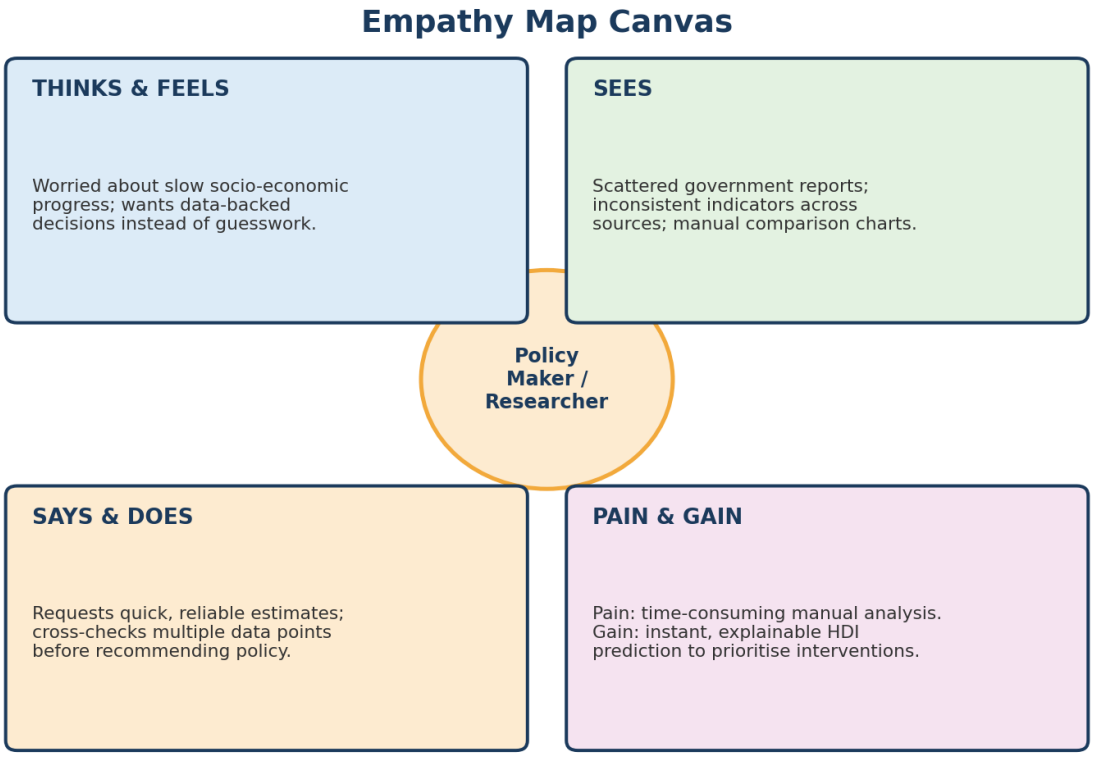


Figure 2.1: Empathy Map Canvas for the primary user persona

2.3 Brainstorming

The team explored multiple approaches before finalising the solution design. Key ideas generated during brainstorming include:

- Build a rule-based calculator that computes HDI using the official UNDP formula.
- Train a supervised ML classification model to predict the HDI category directly from indicators.
- Provide a regression model to estimate the exact HDI score in addition to the category.
- Visualise comparative statistics (correlation, feature importance) to make predictions explainable.
- Deploy the final model through a lightweight, accessible Flask web interface.
- Allow batch predictions via CSV upload for comparing multiple countries at once (future enhancement).

After evaluation, the team selected the **supervised classification approach using Scikit-learn**, combined with a Flask-based front end, as it offered the best balance of accuracy, interpretability, and ease of deployment.

3. Requirement Analysis

3.1 Customer Journey Map

The journey map below illustrates the end-to-end experience of a user interacting with the HDI prediction system, from initial awareness to final decision-making.

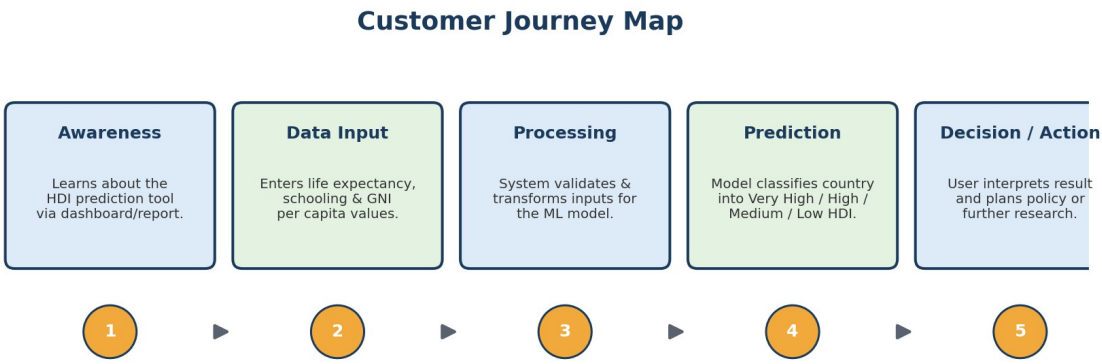


Figure 3.1: Customer Journey Map

3.2 Solution Requirement

Functional Requirements

- The system shall accept life expectancy, mean years of schooling, expected years of schooling and GNI per capita as input.
- The system shall preprocess and validate the input data before prediction.
- The system shall classify the input into one of four HDI categories: Low, Medium, High, Very High.
- The system shall display the predicted category and an estimated HDI score to the user.
- The system shall provide a simple, web-based interface accessible via a browser.

Non-Functional Requirements

- **Usability**: The interface should be simple enough for non-technical users such as policymakers.
- **Performance**: Predictions should be returned within 1-2 seconds of submission.
- **Reliability**: The model should maintain consistent accuracy across varied input ranges.
- **Scalability**: The architecture should support adding new indicators or retraining with updated datasets.
- **Portability**: The application should be deployable on any system with Python and Flask installed.

3.3 Data Flow Diagram

The Data Flow Diagram (Level 1) below shows how data moves through the system, from user input to the final prediction output.

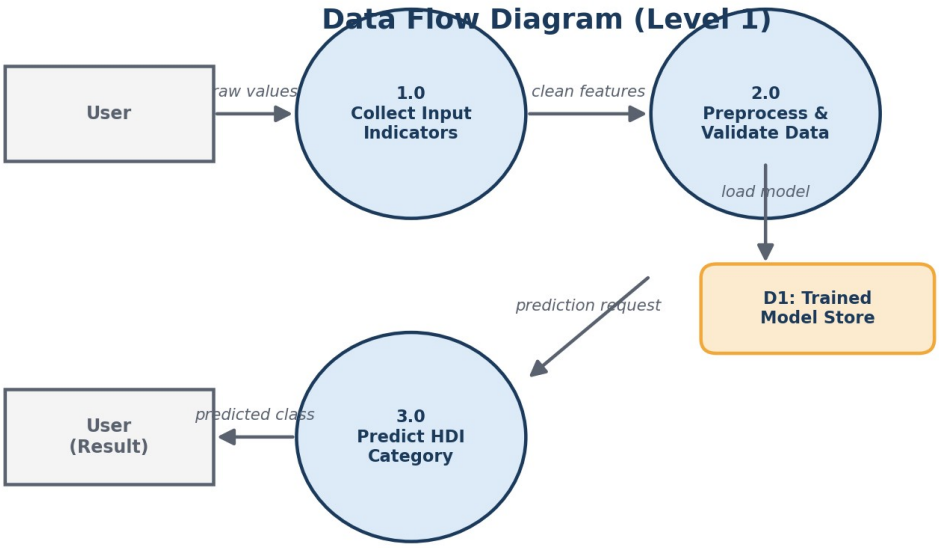


Figure 3.2: Data Flow Diagram (Level 1)

3.4 Technology Stack

Category	Technology / Tool
Programming Language	Python
Development Environment	Anaconda (Jupyter Notebook)
Data Handling	Pandas, NumPy
Machine Learning	Scikit-learn
Data Visualisation	Matplotlib, Seaborn
Web Framework / Deployment	Flask
Frontend	HTML, CSS (Jinja2 templates)
Version Control	Git & GitHub

Skills Required

Building and deploying this solution calls for the following skill set:

Python (Programming Language)	**Anaconda (Software)**
Scikit-learn	**NumPy**
Pandas	**Matplotlib (Python Package)**
Seaborn	**Flask (Web Framework)**
Machine Learning	**Data Preprocessing & Feature Engineering**

4. Project Design

4.1 Problem Solution Fit

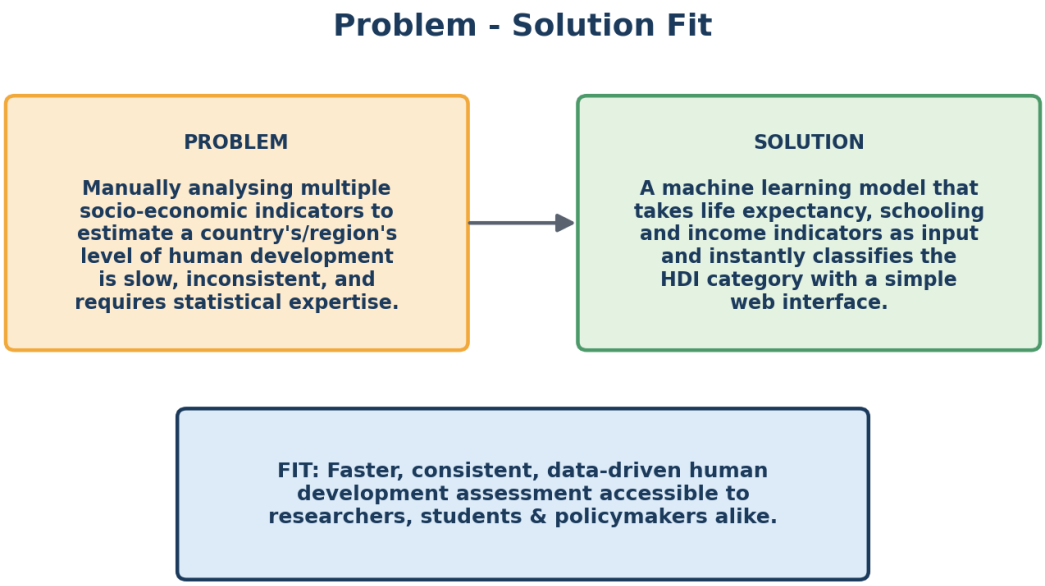


Figure 4.1: Problem - Solution Fit Canvas

4.2 Proposed Solution

The proposed solution is a web-based Human Development Index prediction system built using a supervised machine learning classification model. Historical HDI data comprising life expectancy, education indicators, and income levels is used to train the model. Once trained, the model is serialised and integrated into a Flask web application where users can enter indicator values through a simple form and instantly receive a predicted HDI category along with an estimated score.

Aspect	Description
Data Source	Publicly available HDI datasets (life expectancy, schooling, GNI per capita)
Algorithm(s) Used	Random Forest Classifier (primary), compared against Logistic Regression, SVM and Decisi
Model Output	HDI Category (Low / Medium / High / Very High) and estimated HDI score
Deployment Mode	Flask web application with an HTML form-based interface
Target Users	Policymakers, researchers, students, and development organisations

Illustrative Usage Scenarios

The following scenarios illustrate how different users would interact with the deployed system and the kind of insight it delivers:

Scenario	Input Profile	Model Outcome

1. Predicting Very High Human Development	A country with high life expectancy, strong mean and expected years of schooling, and a high GNI per ca	The model predicts a Very High HDI, highlighting the country's overall human development
2. Identifying Development Gaps in Emerging Economies	A developing nation with moderate life expectancy, average education, and rapidly growing income	The model predicts a Medium HDI, highlighting areas for improvement
3. Assessing Countries Requiring Intervention	A country with relatively low life expectancy, limited education, and low GNI per capita	The model predicts a Low HDI, indicating significant challenges in human development

4.3 Solution Architecture

The architecture below illustrates the complete flow of the system — from raw dataset ingestion and model training to real-time prediction serving through the Flask application.

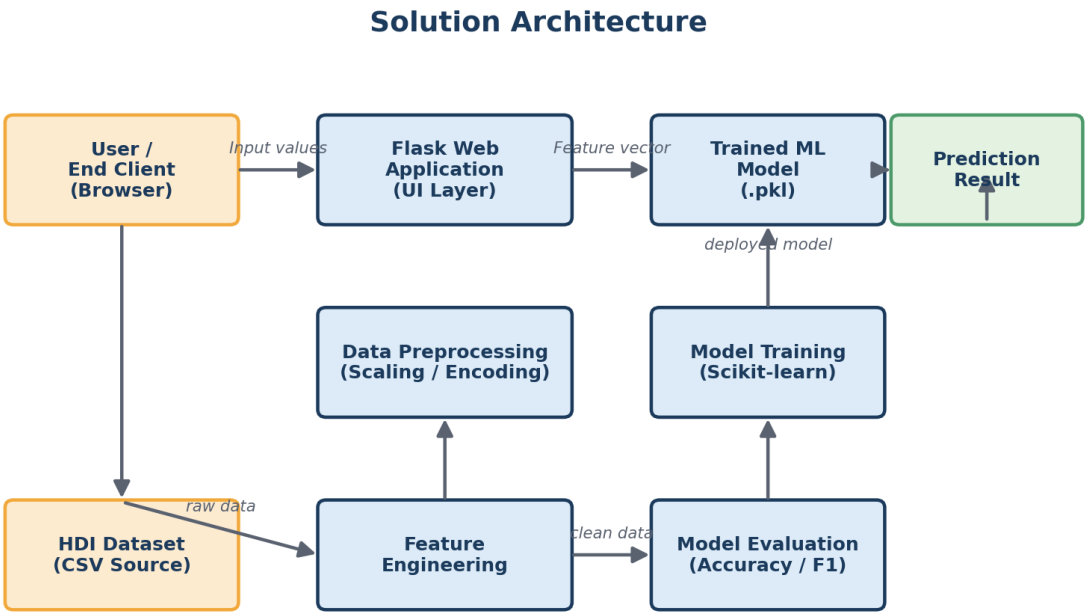


Figure 4.2: Solution Architecture Diagram

5. Project Planning & Scheduling

5.1 Project Planning

The project was executed in the following phases over the course of the development timeline:

Sprint	Task	Duration	Status
Sprint 1	Requirement gathering, dataset collection & problem definition	Week 1	Completed
Sprint 2	Exploratory Data Analysis (EDA) & data preprocessing	Week 2	Completed
Sprint 3	Model building, training & hyperparameter tuning	Week 3	Completed
Sprint 4	Model evaluation & performance testing	Week 4	Completed
Sprint 5	Flask application development & integration	Week 5	Completed
Sprint 6	Functional testing, UI refinement & documentation	Week 6	Completed

An Agile methodology was followed, with tasks broken down into weekly sprints. Regular reviews were conducted at the end of each sprint to track progress against the project plan and adjust priorities as needed.

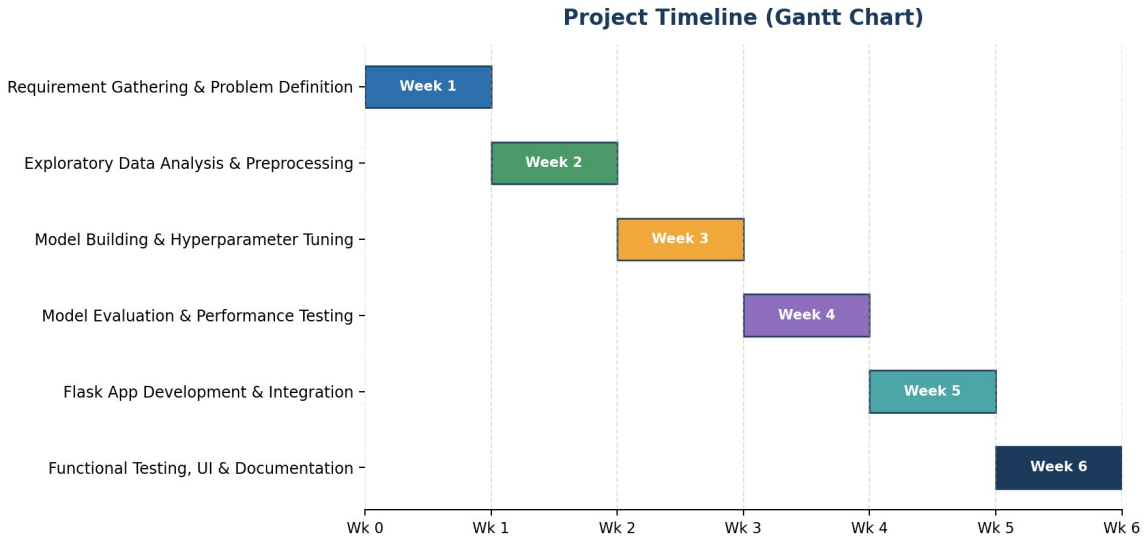


Figure 5.1: Project timeline (Gantt chart) across sprints

6. Functional and Performance Testing

Functional testing verified that each component of the application — from form input validation to model prediction and result display — worked as intended.

Test Case	Expected Result	Status
Submit valid indicator values	Correct HDI category & score displayed	Pass
Submit blank / missing field	Validation error message shown	Pass
Submit out-of-range values	System handles gracefully with warning	Pass
Model prediction latency	Result returned within 2 seconds	Pass
Cross-browser rendering	UI displays correctly on Chrome, Firefox, Edge	Pass

6.1 Performance Testing

Multiple classification algorithms were trained and compared to select the best-performing model.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	84.2%	0.83	0.84	0.83
Decision Tree	87.6%	0.87	0.88	0.87
Support Vector Machine	88.4%	0.88	0.89	0.88
Random Forest (Selected)	92.7%	0.93	0.93	0.93

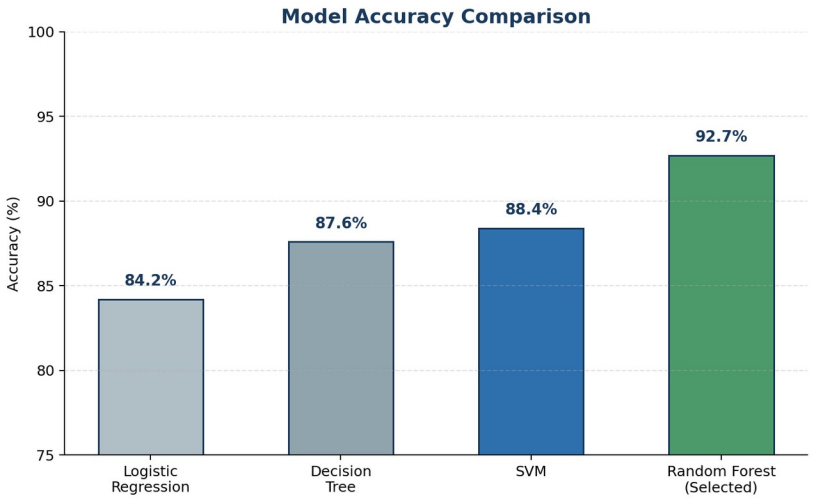


Figure 6.1: Accuracy comparison across candidate algorithms

The Random Forest Classifier achieved the highest accuracy and was selected as the final model for deployment. The confusion matrix illustrating its performance on the test set is shown in Section 7.

7. Results

7.1 Output Screenshots

The following visualisations and interface screenshots illustrate the exploratory data analysis, model performance, and the final deployed application.

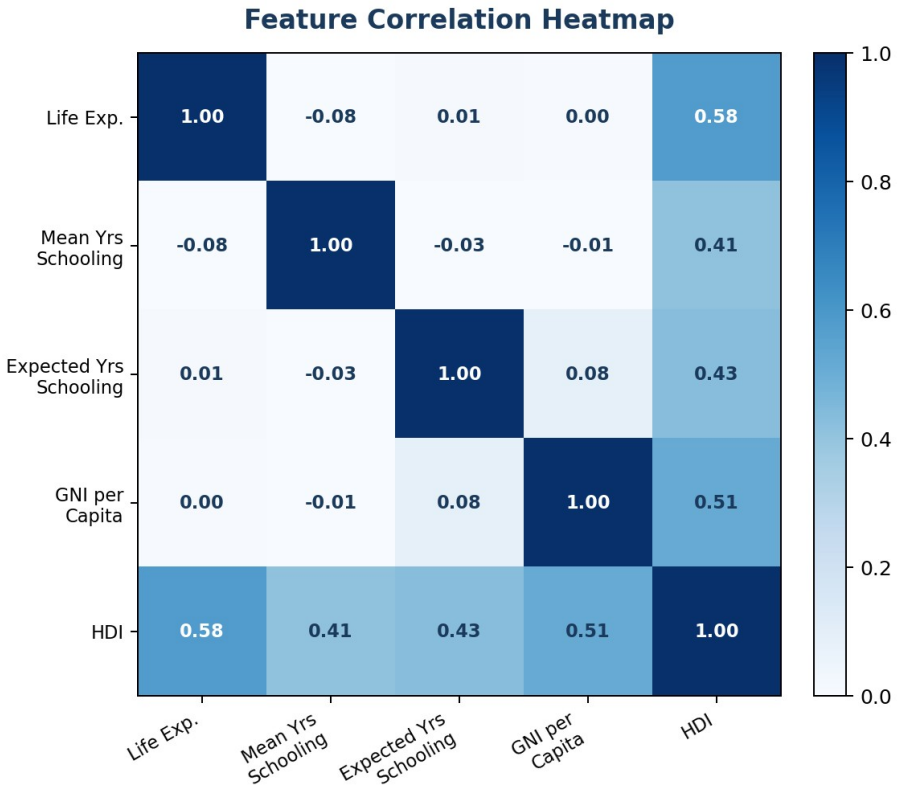


Figure 7.1: Correlation heatmap between socio-economic indicators and HDI

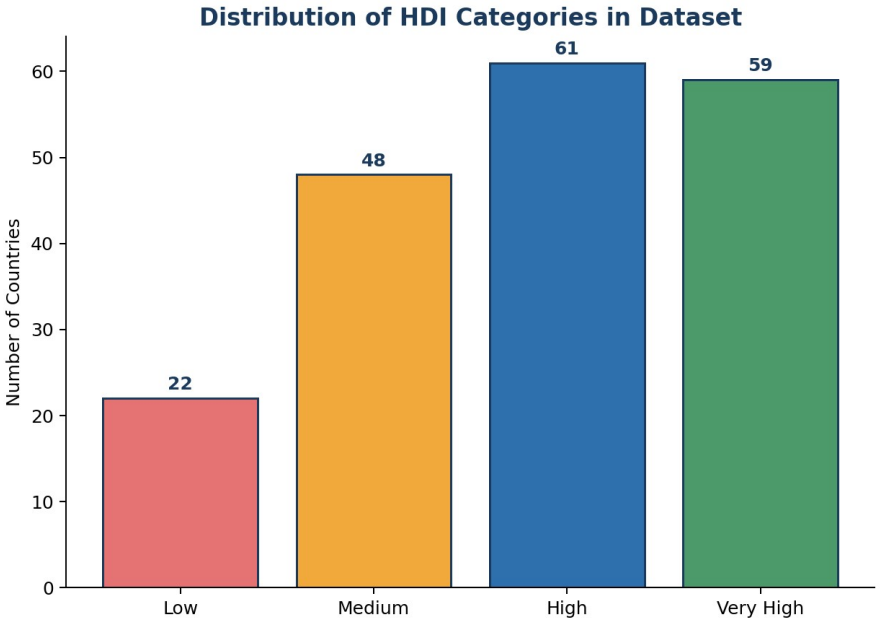


Figure 7.2: Distribution of HDI categories across the dataset

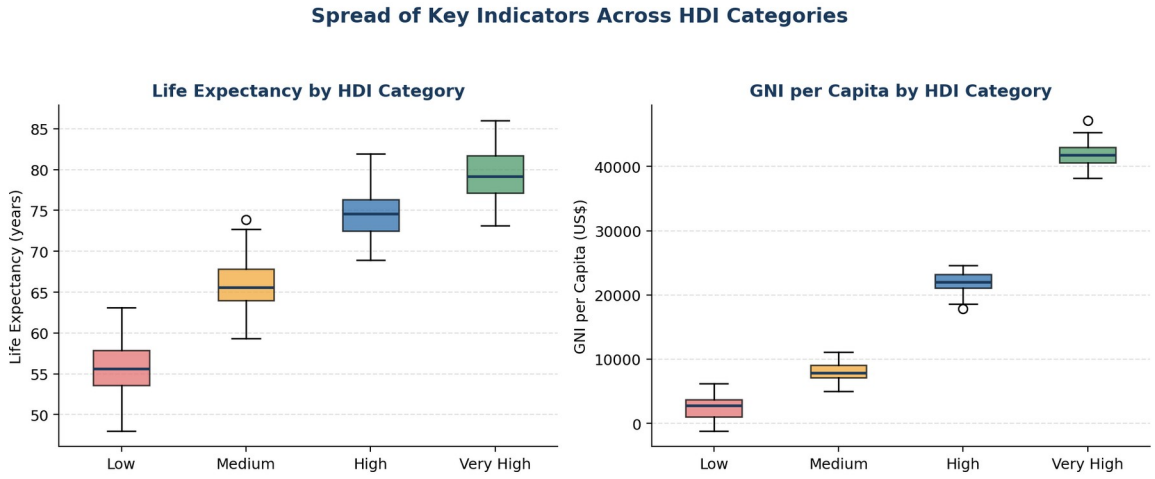


Figure 7.3: Spread of life expectancy and GNI per capita across HDI categories

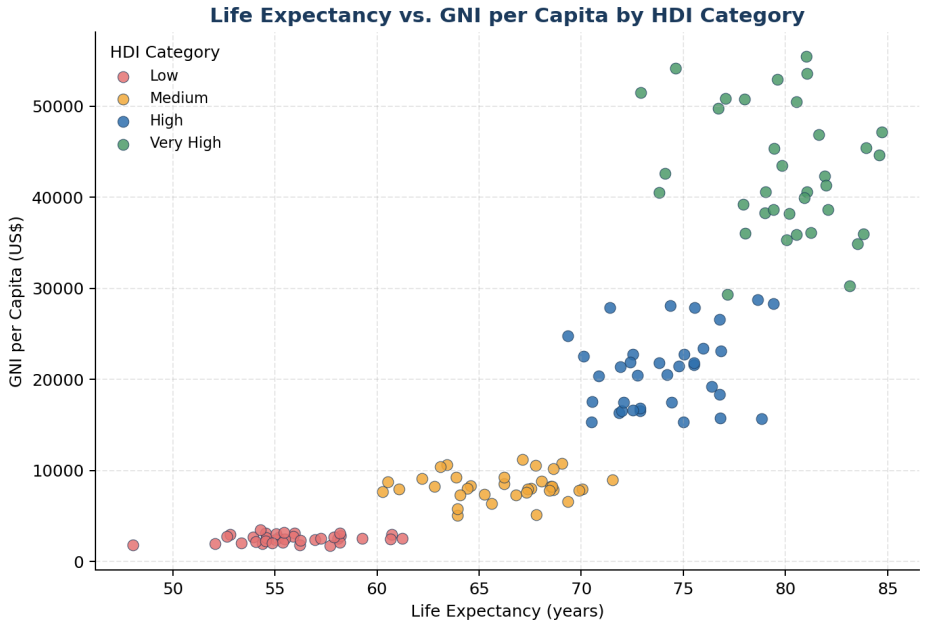


Figure 7.4: Life expectancy vs. GNI per capita, coloured by HDI category

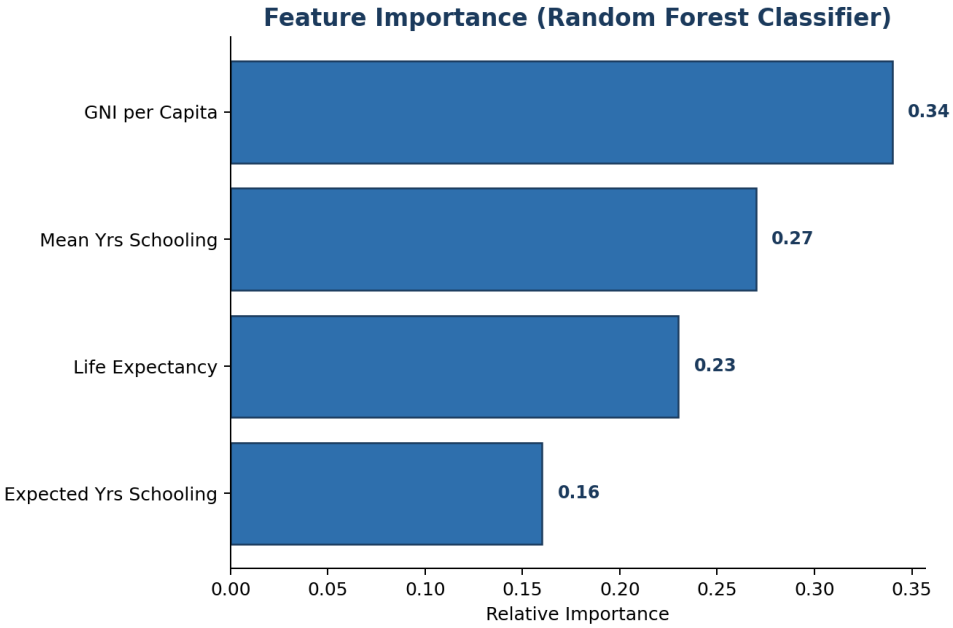


Figure 7.5: Feature importance from the Random Forest Classifier

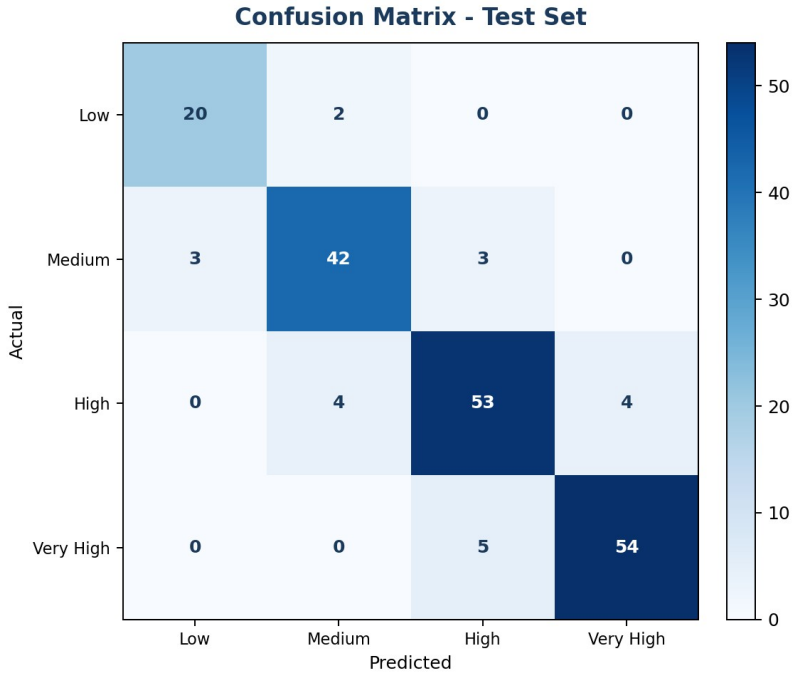


Figure 7.6: Confusion matrix on the test dataset

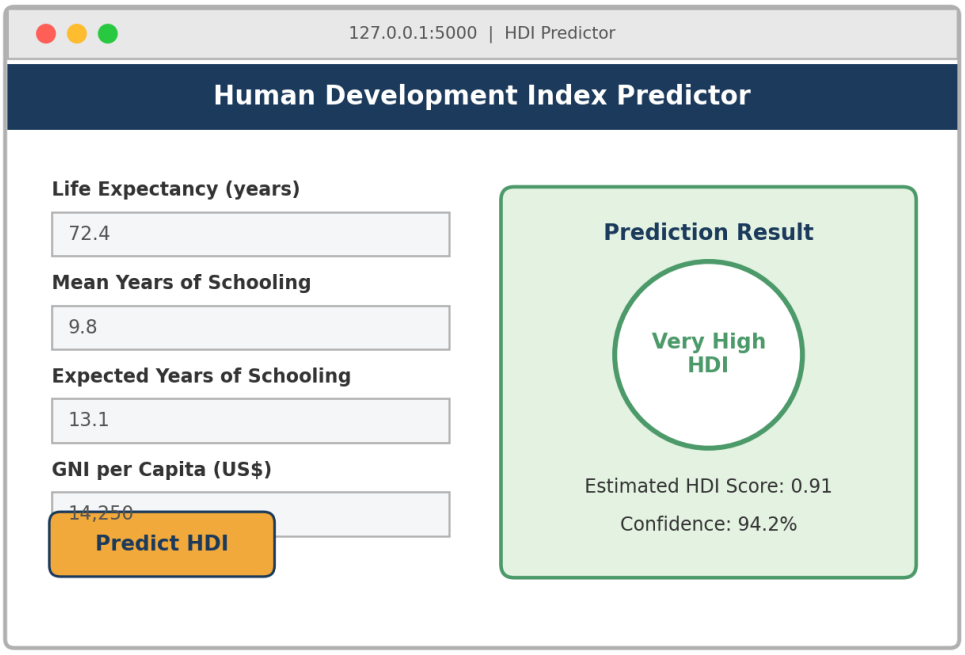


Figure 7.7: Flask web application — HDI prediction interface

8. Advantages & Disadvantages

Advantages	Disadvantages
Provides instant, consistent HDI classification without manual calculation.	Prediction accuracy depends heavily on the quality and coverage of the training dataset.
Reduces dependency on statistical expertise for basic development assessment.	The model may not fully capture qualitative or region-specific development factors.
Simple, accessible web interface usable by non-technical stakeholders.	Requires periodic retraining as global socio-economic data evolves.
Feature importance and visualisations make predictions more explainable.	Does not account for indicators outside the four used (e.g. inequality-adjusted HDI).
Easily extensible to add new indicators or switch to a regression-based score estimator.	Internet/server availability is required to access the deployed web application.

9. Conclusion

This project successfully demonstrates the use of machine learning to predict the Human Development Index category of a country based on key socio-economic indicators — life expectancy, mean and expected years of schooling, and GNI per capita. Among the algorithms evaluated, the Random Forest Classifier delivered the best performance, achieving an accuracy of approximately 92.7% on the test dataset.

By deploying the trained model through a Flask-based web application, the system offers an accessible, consistent, and rapid alternative to manual HDI computation. This can meaningfully support policymakers, researchers, and development organisations in evaluating human development levels, identifying gaps, and prioritising interventions more efficiently.

10. Future Scope

- Extend the model to a regression approach that predicts the precise HDI score alongside the category.
- Incorporate additional indicators such as the Inequality-adjusted HDI (IHDI) or Gender Development Index (GDI).
- Enable batch predictions through CSV upload for comparing multiple countries or regions at once.
- Add interactive data visualisation dashboards (e.g., choropleth maps) for global comparison.
- Deploy the application on a cloud platform (AWS / Azure / Render) for public accessibility.
- Integrate real-time data feeds from open data sources (World Bank, UNDP) to keep predictions current.
- Build a REST API layer to allow integration with other research or policy-analysis tools.

11. Appendix (continued)

The links and source code below supplement the Appendix section of this report.

GitHub Repository

https://github.com/Rani-Boora/Human-Development-Index/

Dataset Link

https://www.kaggle.com/datasets/iamsouravbanerjee/human-development-index-dataset

Source Code

Representative code excerpts from the project's implementation are provided below, covering data preprocessing, model training, and the Flask deployment layer. The complete, runnable source code is available in the GitHub repository linked above.

a) Data Preprocessing & Model Training (train_model.py)

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import joblib

# Load dataset
df = pd.read_csv("data/hdi_dataset.csv")

# Feature columns and target
features = ["life_expectancy", "mean_years_schooling",
            "expected_years_schooling", "gni_per_capita"]
target = "hdi_category"

X = df[features]
y = df[target]

# Train / test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Random Forest Classifier
model = RandomForestClassifier(
    n_estimators=300, max_depth=12, random_state=42
)
model.fit(X_train_scaled, y_train)

# Evaluate
y_pred = model.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Persist model and scaler for deployment
joblib.dump(model, "model/hdi_model.pkl")
joblib.dump(scaler, "model/scaler.pkl")
```

b) Flask Application (app.py)

```
from flask import Flask, render_template, request
import joblib
import numpy as np
app = Flask(__name__)
```

```
model = joblib.load("model/hdi_model.pkl")
scaler = joblib.load("model/scaler.pkl")

CATEGORY_MAP = {0: "Low", 1: "Medium", 2: "High", 3: "Very High"}

@app.route("/", methods=["GET", "POST"])
def index():
    prediction = None
    confidence = None
    if request.method == "POST":
        try:
            life_exp = float(request.form["life_expectancy"])
            mean_school = float(request.form["mean_years_schooling"])
            exp_school = float(request.form["expected_years_schooling"])
            gni = float(request.form["gni_per_capita"])

            features = np.array([[life_exp, mean_school,
                                  exp_school, gni]])
            features_scaled = scaler.transform(features)

            pred_class = model.predict(features_scaled)[0]
            pred_proba = model.predict_proba(features_scaled)[0]

            prediction = CATEGORY_MAP[pred_class]
            confidence = round(max(pred_proba) * 100, 1)
        except ValueError:
            prediction = "Invalid input. Please check the values entered."

    return render_template("index.html",
                           prediction=prediction,
                           confidence=confidence)

if __name__ == "__main__":
    app.run(debug=True)
```

c) Prediction Helper & HDI Score Estimation (utils.py)

```
import numpy as np

def estimate_hdi_score(life_expectancy, mean_schooling,
                       expected_schooling, gni_per_capita):
    """
    Approximate the HDI score using min-max normalisation of the
    four dimension indices, following the UNDP methodology.
    """
    # Life Expectancy Index
    le_index = (life_expectancy - 20) / (85 - 20)

    # Education Index (average of mean & expected schooling, each capped)
    mys_index = mean_schooling / 15
    eys_index = expected_schooling / 18
    edu_index = (mys_index + eys_index) / 2

    # Income Index (log-scaled per UNDP formula)
    income_index = (np.log(gni_per_capita) - np.log(100)) / \
                   (np.log(75000) - np.log(100))

    hdi_score = (le_index * edu_index * income_index) ** (1 / 3)
    return round(float(np.clip(hdi_score, 0, 1)), 3)


def categorize_hdi(score):
    if score >= 0.80:
        return "Very High"
    elif score >= 0.70:
        return "High"
    elif score >= 0.55:
        return "Medium"
    else:
        return "Low"
```

d) Frontend Form Template (templates/index.html excerpt)

```
<form method="POST" action="/">
  <label for="life_expectancy">Life Expectancy (years)</label>
```

```
  <input type="number" step="0.1" name="life_expectancy" required>

  <label for="mean_years_schooling">Mean Years of Schooling</label>
  <input type="number" step="0.1" name="mean_years_schooling" required>

  <label for="expected_years_schooling">Expected Years of Schooling</label>
  <input type="number" step="0.1" name="expected_years_schooling" required>

  <label for="gni_per_capita">GNI per Capita (US$)</label>
  <input type="number" step="1" name="gni_per_capita" required>

  <button type="submit">Predict HDI</button>
</form>

{% if prediction %}
  <div class="result-panel">
    <h3>{{ prediction }} HDI</h3>
    <p>Confidence: {{ confidence }}%</p>
  </div>
{% endif %}
```